Fraenkel-Mostowski atoms model *variables* as well as names

Murdoch J. Gabbay[1]

School of Computer Science
Heriot-Watt University
EDINBURGH EH14 4AS, Scotland
murdoch.gabbay@gmail.com

Abstract. In previous work we have shown how Fraenkel-Mostowski set theory can deliver a good model of *names* and their abstraction. In this paper we show that this model extends to a model of *variables* and their substitution, by exhibiting substitution for names in Fraenkel-Mostowski sets, as an operation on sets. Furthermore, this operation is capture-avoiding in the sense that it interacts properly with the notion of abstraction from previous work. We explore some intriguing implications of the existence of a syntax-like 'substitution action' on a set-theoretic universe.

1 Introduction

Fraenkel-Mostowski sets [5, 22] provide a model of naming and abstraction; in the case of sets representing abstract syntax these notions of name and abstraction coincide with the usual notions of 'free name' and 'name-binding' in syntax [13]. Fraenkel-Mostowski set theory can model more than syntax though; it is a foundational theory and the model of naming and abstraction which it provides, has inspired programming languages with datatypes for names and constructors to bind those names [18, 6].

In this paper we demonstrate by a non-trivial but elementary construction on sets, that Fraenkel-Mostowski set theory is also a model of capture-avoiding substitution on names. A name with a capture-avoiding substitution action is a model of 'a variable', so this paper shows that the Fraenkel-Mostowski model of naming extends to a model of variables.

We shall show that in the case of sets representing abstract syntax, the substitution action coincides with the 'usual' substitution defined inductively on syntax. We show that on *all* sets — not just those representing syntax — the action satisfies the conditions laid out by an algebraic axiomatisation of substitution developed in previous work [12]; and we explore how one might generalise that axiomatisation to the far richer structure such as a sets universe.

Usual notions of denotation deny variables any denotational reality; a variable in the syntax *ranges over* denotational elements but is not *in* the denotation. Put another way, 'x' can represent any denotational element but there is no single denotational element we can point to and say 'this represents the variable x'.

Our construction of substitution justifies a view according to which variables are denotational objects in the sets universe with equal status with other elements of the

model like numbers, groups, abstract syntax, functions, and everything else that a foundational theory can express. The substitution action on the universe lets us replace these denotation objects in a sensible way.

Map of the paper. In Section 2 we consider basic properties of FM sets (full proofs are elsewhere [13]). In Subsection 3.1 we consider what properties 'a substitution action' should possess, in the light of previous work. In Subsection 3.2 we consider some *unsatisfactory* substitution actions and use them to motivate some of the very technical aspects of our construction. In Subsection 3.3 we give the construction itself and prove its properties. In Subsection 3.4 we consider in detail one of the commutativity properties of substitution. In Section 4 we consider related work and conclusions.

2 Fraenkel-Mostowski sets

Fraenkel-Mostowski set theory **(FM sets)** is a variant of Zermelo-Fraenkel set theory **(ZF sets)**, originally introduced to prove the axiom of choice from the other axioms of ZF sets [5, 22].

FM sets is extremely similar to ZF sets. The main difference is that in FM sets we assume a set of **atoms** $\mathbb{A}$. Atoms are 'empty sets' in the sense that if $a \in \mathbb{A}$ then $\forall x.x \notin a$. However atoms are not equal to the empty set $\emptyset$.[1] Also, an axiom insists that there be infinitely many atoms.

The reader can think of a typical FM set as being an atom, or a set containing nothing but atoms or more sets. For full details see elsewhere [10].

As a matter of convention for the rest of this paper:

- *The permutative convention:* $a, b, c, \ldots$ range over *distinct* atoms unless stated otherwise.
- A, B, C, S, T range over sets of atoms. For example $A \subseteq \mathbb{A}$.
- X, Y, Z, U, V range over elements that are not atoms and may be empty. For example X might equal $\emptyset$ or $\{a, \emptyset\}$, but X cannot equal a.
- x, y, z, u, v range over arbitrary elements.

Write $(a\ b)$ for the **swapping** of a and b. This is the mapping from $\mathbb{A}$ to itself specified by

$$(a\ b)(a) = b \qquad (a\ b)(b) = a \qquad (a\ b)(c) = c.$$

The action of swappings extends to all FM sets by ϵ-induction [16]:

$$(a\ b)X = \{(a\ b)x \mid x \in X\}.$$

Write $\circ$ for functional composition and let π range over functions from $\mathbb{A}$ to $\mathbb{A}$ generated by functionally composing finitely many swappings. The action of π on the FM set universe is the **permutation action**.

Let $\phi(x_1, \ldots, x_n)$ range over predicates in the language of FM sets (which is just a first-order logic with two binary symbols $=$ for equality and $\in$ for set inclusion).

[1] In ZF sets the axiom of extensionality decrees that any two empty sets must be equal; in FM sets the axiom of extensionality is weakened so that any empty set must either be equal to *the* empty set $\emptyset$ — or be an atom.

Suppose that ϕ mentions variables in x_1, ..., x_n. Note in passing that a function $F(x_1, \ldots, x_n)$ can be expressed using a predicate $\phi_F(x_1, \ldots, x_n, z)$ such that for given x_1,..., x_n there is a unique z making ϕ_F true.

Theorem 2.1. *$\phi(x_1, \ldots, x_n) \Leftrightarrow \phi(\pi x_1, \ldots, \pi x_n)$, and*
$\pi(F(x_1, \ldots, x_n)) = F(\pi x_1, \ldots, \pi x_n)$
always hold; we call this property **equivariance** *(of the language of FM sets).*

Proof. The first part is by an easy induction on the syntax of ϕ. We consider just one case: $x \in y$ implies $\pi x \in \pi y$ follows directly from the fact that $\pi y = \{\pi y' \mid y' \in y\}$. The reverse implication uses π^{-1}.

The second part, for F, follows using the encoding of functions as predicates. □

Write

$$a\#x \qquad \text{when} \qquad \exists S.S \in \mathcal{P}_{fin}\mathbb{A} \wedge \forall b.(b \notin S \Rightarrow (b\ a)x = x).$$

(Here $\mathcal{P}_{fin}\mathbb{A}$ represents the set of finite sets of atoms.) Read this as a **is fresh for** x [13]. In words this says:

> "a is fresh for x when there is some finite set of atoms such that any permutation fixing that set pointwise must fix x — and a is not in that set."

We may write $a\#t_1, t_2$ for '$a\#t_1$ and $a\#t_2$', and so on. Write

$$supp(x) \qquad \text{for} \qquad \{a \in \mathbb{A} \mid \neg(a\#x)\}.$$

Call this the **support** of x.

Suppose $A \subseteq \mathbb{A}$. Write $\mathsf{fix}(A)$ for the set of permutations π such that $\forall a \in A.\pi(a) = a$. Say that $S \subseteq \mathbb{A}$ **supports** x when

$$\forall \pi \in \mathsf{fix}(S).\pi x = x.$$

Theorem 2.2. *$supp(x)$ is the unique smallest set supporting x. Furthermore if $a\#x$ for all a such that $\pi(a) \neq \pi'(a)$ then $\pi x = \pi' x$.*

The meaning of Theorem 2.2 is that $supp(x)$ is the unique smallest S we can use in the definition of $a\#x$ above. Furthermore, the action of π on x depends *only* on its action on $supp(x)$.

The proofs of these properties non-trivial and we must refer the reader to previous work — which is now well-known and thoroughly checked over — for details [13, 10].

Theorem 2.3. *S supports x if and only if πS supports πx. As a corollary, $\pi supp(x) = supp(\pi x)$.*

Proof. From Theorem 2.1. □

Theorem 2.4. *$supp(F(x_1, \ldots, x_n)) \subseteq supp(x_1) \cup \cdots \cup supp(x_n)$.*

Proof. Suppose $S = supp(x_1) \cup \ldots \cup supp(x_n)$. By Theorem 2.1 $\pi F(x_1, \ldots, x_n) = F(\pi x_1, \ldots, \pi x_n)$. If $\pi \in \mathsf{fix}(S)$ then $\pi \in \mathsf{fix}(x_1) \cap \ldots \cap \mathsf{fix}(x_n)$ and the result follows. □

For detailed proofs see elsewhere [13, 10].

3 The substitution action

We recall our notational convention that a, b, c range over distinct atoms, A, B, C, S, T range over sets of atoms, lowercase x, y, z, u, v range over all elements, and uppercase X, Y, Z, U, V range over elements that are not atoms.

3.1 The axioms

A **substitution action** on FM sets is a function $z[a{\mapsto}x]$ expressed in the language of FM sets (and so satisfying equivariance Theorem 2.1) taking any z, an atom a, and any x, satisfying:

$$
\begin{array}{rl}
(\alpha) & b\#z \Rightarrow z[a{\mapsto}x] = ((b\ a)z)[b{\mapsto}x] \\
(\#{\mapsto}) & a\#z \Rightarrow z[a{\mapsto}x] = z \\
(\mathbf{var}{\mapsto}) & a[a{\mapsto}x] = x \\
(\mathbf{ren}{\mapsto}) & b\#z \Rightarrow z[a{\mapsto}b] = (b\ a)z \\
(\mathbf{abs}{\mapsto}) & c\#x \Rightarrow ([c]z)[a{\mapsto}x] = [c](z[a{\mapsto}x]) \\
(\mathbf{F}{\mapsto}) & F(x_1,\ldots,x_n)[a{\mapsto}x] = F(x_1[a{\mapsto}x],\ldots,x_n[a{\mapsto}x])
\end{array}
$$

This does not define a particular substitution on FM sets; that comes later. These are simply properties we require of the construction.

Here by our convention x, y, z range over elements of the Fraenkel-Mostowski sets universe and a, b, and c range over distinct atoms. F ranges over all functions — more on that later. $[c]z$ is the Gabbay-Pitts *abstraction* operator — more on that later (a definition is in Definition 3.1 or previous work [13]).

These axioms are taken from work with Mathijssen [12] in a syntax-based context (so freshness $\#$ is a notion of 'free atoms of' and x, y, z range over terms). There it was proved that these axioms are sound and complete with respect to a closed term model where substitution is interpreted by a syntax-based 'capture-avoiding substitution function'. Please note that we proved they are *complete*. So for syntax-with-binding, the axioms of substitution above capture *precisely* the properties of 'capture-avoiding substitution'.[2]

A model of FM sets is far larger (and richer) than any set of syntax trees. It nevertheless supports interesting and useful generalisations of the notion of 'name', 'freshness', and 'abstraction'. Some time ago the author commented on the potential to extend this apparatus to a notion of substitution [10].

A significant difference between the axioms above and the axioms of [12] is that there, the 'F' in $(\mathbf{F}{\mapsto})$ ranges over term-formers of a fixed but arbitrary signature. This was appropriate in a syntactic development. Here F should refer to functions on sets. But the language of FM sets can express a great many F so we must read $(\mathbf{F}{\mapsto})$ carefully — more on that later. Other differences are unimportant.[3]

[2] Other axiomatisations of substitution exist, but not to our knowledge of *capture-avoiding* substitution. See the Conclusions.

[3] (α) appears in [12], but indirectly — it is guaranteed by *other* parts of the Nominal Algebra framework which that paper uses; also the `var` term-former of [12] is an artefact of the typing system which we use in that paper and it vanishes here because sets are untyped.

3.2 Candidate substitution actions

In the next subsection we define our substitution action. First, we take a moment to motivate it with some unsatisfactory substitution actions. The **naïve** substitution action

$$a[a{\mapsto}x]_n = x \qquad b[a{\mapsto}x]_n = b \qquad Z[a{\mapsto}x]_n = \{z[a{\mapsto}x]_n \mid z \in Z\}$$

is simple but wrong in the sense that it does not satisfy (α), $(\#{\mapsto})$, $(\mathbf{ren}{\mapsto})$, or $(\mathbf{abs}{\mapsto})$; whether it satisfies $(\mathbf{F}{\mapsto})$ depends on the F. For example $\mathbb{A}[a{\mapsto}\emptyset]_n = (\mathbb{A} \setminus \{a\}) \cup \{\emptyset\} \neq \mathbb{A}$ but $a\#\mathbb{A}$, contradicting $(\#{\mapsto})$.

So the naïve action is incompatible with the Fraenkel-Mostowski sets treatment of names and abstraction.

To define the next substitution action we need machinery. Suppose $A \subseteq \mathbb{A}$. Write

$$u\|_A \quad \text{for} \quad \{\pi u \mid \pi \in \mathsf{fix}(A)\}.$$

(Recall that $\mathsf{fix}(A) = \{\pi \mid \forall a \in A.\pi(a) = a\}$.) Note that $u\|_\emptyset = \{\pi u \mid \text{all } \pi\}$. Write (x, y) for the pairset. This can be implemented as $\{\{x\}, \{x, y\}\}$ or in any number of other ways; this is standard [16]. So examples are:

$$\{(a, b)\}\|_{\{a,c\}} = \{(a, y) \mid y \in \mathbb{A} \setminus \{a, c\}\} \qquad \{a\}\|_\emptyset = \mathbb{A}$$
$$\{b\}\|_{\{a\}} = \mathbb{A} \setminus \{a\} \qquad \{a\}\|_{\{a\}} = \{a\}.$$

Definition 3.1. *Define the* **atoms abstraction** *by* $[c]z = (c, z)\|_{supp(z)\setminus\{c\}}$.

It is not hard to verify that $supp([c]z) = supp(z) \setminus \{c\}$. It is also not hard to verify that this definition is consistent with the definition $[c]z = \{(n, (n\ c)z) \mid n = c \vee n\#z\}$ used in the original paper for binding names [13].

Then the **trivial** substitution action is defined by:

$$a[a{\mapsto}x]_t = x \qquad b[a{\mapsto}x]_t = b \qquad Z[a{\mapsto}x]_t = Z \quad (a\#Z)$$
$$Z[a{\mapsto}b]_t = (b\ a)Z \quad (b\#Z) \qquad ([c]z)[a{\mapsto}x]_t = [c](z[a{\mapsto}x]_t) \quad (c\#x)$$
$$Z[a{\mapsto}x]_t = \{z[a{\mapsto}x]_t \mid z \in Z\} \quad (Z \text{ finite}) \qquad Z[a{\mapsto}x]_t = \emptyset \quad (\text{otherwise}).$$

This satisfies the axioms of substitution (it is surprising that nobody has commented on this before even in passing; the vocabulary has been in place for several years [13]).

But we should also look at *how* $[a{\mapsto}x]_t$ satisfies the axioms of substitution; this is clearly capture-avoiding substitution on the subclass of FM sets which represent abstract syntax trees — but for more complex sets the action 'gives up' and defaults to $\emptyset$. So it teaches us little about substitution and the rest of the FM sets universe.

The problem is that the axioms we inherit from previous work are designed for syntax and are too weak to pin down a unique substitution action in a universe full of complex and infinitary stucture, as FM sets is. We now define a more 'sets-like' substitution action; more on the issue of canonicity later.

3.3 The construction

Call (u, A) a **plane** in Z when $u \in Z$ and $A = \mathit{supp}(u\|_\emptyset \cap Z)$. Write $\mathsf{plane}(Z)$ for the set of planes in Z. For example if $a, b \in \mathbb{A}$ then:

$$(a, \{a\}) \in \mathsf{plane}(\{a\}) \qquad (a, \emptyset) \in \mathsf{plane}(\mathbb{A}) \qquad (b, \{a\}) \in \mathsf{plane}(\mathbb{A}\backslash\{a\})$$
$$(a, \{b\}),\ ((b, b), \{\}) \ \in\ \mathsf{plane}\big((\mathbb{A} \setminus \{b\}) \cup \{(x, x) \mid x \in \mathbb{A}\}\big)$$
$$((a, c), \{a, b\}),\ ((a, c), \{b, c\}) \ \in\ \mathsf{plane}(\{(x, y) \mid (x{=}a \wedge y{\neq}b) \vee (x{\neq}b \wedge y{=}c)\}).$$

Lemma 3.2. *If $(u, A) \in \mathsf{plane}(Z)$ then $u\|_A \subseteq Z$.*

Proof. By assumption $u \in Z$ and $A = \mathit{supp}(u\|_\emptyset \cap Z)$ and $u\|_A = \{\pi u \mid u \in \mathsf{fix}(A)\}$. So suppose $\pi \in \mathsf{fix}(A)$. Then $\pi u \in \pi(u\|_\emptyset \cap Z)$ by Theorem 2.1. By Theorem 2.2 also $\pi(u\|_\emptyset \cap Z) = u\|_\emptyset \cap Z$. It follows that $\pi u \in Z$. □

So we should think of $(u, A) \in \mathsf{plane}(Z)$ as determining a subset of Z 'swept out by u'.

If $S \in \mathcal{P}_{\mathit{fin}}\mathbb{A}$ then define

$$\mathsf{plane}_S(Z) = \{(u, A) \in \mathsf{plane}(Z) \mid \mathit{supp}(u) \cap S \subseteq \mathit{supp}(u) \cap A\}.$$

We should think of $\mathsf{plane}_S(Z)$ as the planes (u, A) in Z such that $\mathit{supp}(u)$ 'avoids clashes' with atoms in S, where this is possible. For example

$$(c, \{a\}) \in \mathsf{plane}_{\{b\}}(\mathbb{A} \setminus \{a\}) \quad \text{but} \quad (b, \{a\}) \notin \mathsf{plane}_{\{b\}}(\mathbb{A} \setminus \{a\}).$$

Lemma 3.3. $\bigcup\{u\|_A \mid (u, A) \in \mathsf{plane}(Z)\} = Z$.
Also for any $S \in \mathcal{P}_{\mathit{fin}}\mathbb{A}$, $\bigcup\{u\|_A \mid (u, A) \in \mathsf{plane}_S(Z)\} = Z$.

Proof. For the first part, note that if $u \in Z$ then $u \in u\|_A$ where $A = \mathit{supp}(u\|_\emptyset \cap Z)$.

For the second part, suppose $u \in Z$. There is a $\pi \in \mathsf{fix}(A)$ which 'freshens' the atoms in $\mathit{supp}(u)$ so that they are disjoint from those in A. By Theorem 2.3 $\mathit{supp}(\pi u) = \pi\mathit{supp}(u)$. Since $\mathsf{fix}(A)$ is a group we know $(\pi u)\|_A = u\|_A$, so $u \in (\pi u)\|_A$. The result follows. □

Definition 3.4. *Define the* **substitution action** *$z[a{\mapsto}x]$ and a 'helper' function $\delta(z, a, x)$ by ϵ-induction as follows:*

- $a[a{\mapsto}x] = x$.
- $b[a{\mapsto}x] = b$.
- *If $Z \notin \mathbb{A}$ then*

$$Z[a{\mapsto}x] = \bigcup\{(u[a{\mapsto}x])\|_{A(a\mapsto x)\setminus\delta(u,a,x)} \mid (u, A) \in \mathsf{plane}_{\mathit{supp}(x)\cup\{a\}}(Z)\}$$
$$\delta(Z, a, x) = (\mathit{supp}(Z)(a{\mapsto}\mathit{supp}(x))) \setminus \mathit{supp}(Z[a{\mapsto}x]).$$

We consider some examples:

1. $\{a\} = a\|_{\{a\}}$ and $\{a\}[a{\mapsto}x] = x\|_{\mathit{supp}(x)\setminus\emptyset} = \{x\}$.

2. $\mathbb{A}$ can be written as $a\|_{\emptyset}$, but this is not suitable for calculating $\mathbb{A}[a{\mapsto}x]$ because $supp(a) \cap (supp(x) \cup \{a\}) \not\subseteq supp(a) \cap \emptyset$. This is an 'accidental name-clash' between a the representative of $\mathbb{A}$ which we chose, and a the atom we substitute for.
3. $\mathbb{A} = b\|_{\emptyset}$ and $\mathbb{A}[a{\mapsto}x] = b\|_{\emptyset} = \mathbb{A}$.
4. $\mathbb{A}\backslash\{a\} = b\|_{\{a\}}$. Then $(\mathbb{A}\backslash\{a\})[a{\mapsto}x] = b\|_{supp(x)\backslash\emptyset} = \mathbb{A} \setminus supp(x)$.
5. $\mathbb{A}\backslash\{a,b\} = c\|_{\{a,b\}}$. Then $(\mathbb{A}\backslash\{a,b\})[a{\mapsto}x] = c\|_{\{b\}\cup supp(x)} = \mathbb{A}\backslash(\{b\}\cup supp(x))$.
6. $\{(a,c),(a,d),(a,e),\ldots\}{=}(a,c)\|_{\{a,b\}}$. It is a fact that $(a,c)[a{\mapsto}x]{=}(x,c)$. Then $\{(a,c),(a,d),(a,e),(a,f)\}[a{\mapsto}x] = (x,c)\|_{supp(x)\cup\{b\}}$.
7. $\{a,(a,a),(c,c),(d,d),\ldots\}[a{\mapsto}(b,b)] = (a\|_{\{a\}} \cup (d,d)\|_{\{b\}})[a{\mapsto}(b,b)]$
$= \{(b,b)\} \cup (d,d)\|_{\{b\}} = (d,d)\|_{\emptyset}$
$= \{(a,a),(b,b),(c,c),\ldots\}$

   Write $u = \{a,(a,a),(c,c),(d,d),\ldots\}$. Note that

$$supp(u[a{\mapsto}(b,b)]) = \emptyset \quad \text{and} \quad supp(u)(a{\mapsto}supp((b,b))) = \{b\}$$

   so that $\delta(u,a,(b,b)) \neq \emptyset$.

In the calculations above we use the following easy lemma:

Lemma 3.5. *$supp((x,y)) = supp(x) \cup supp(y)$.*

The final example illustrates the use of $\delta(u,a,x)$. It can be that the substitution $[a{\mapsto}x]$ identifies some parts of u so that $supp(u[a{\mapsto}x])$ 'loses' some atoms (b, in the example above). This cannot happen in a syntax tree; it is in some sense too small. $\delta(u,a,x)$ detects those atoms.

Now we show how this construction satisfies the axioms (α), $(\#{\mapsto})$, $(\mathbf{var}{\mapsto})$, $(\mathbf{ren}{\mapsto})$, $(\mathbf{abs}{\mapsto})$, and $(\mathbf{F}{\mapsto})$.

Lemma 3.6. *If $A \in \mathcal{P}_{fin}\mathbb{A}$ and $a \in \mathbb{A}$ then $a\#A$ if and only if $a \notin A$.*

Lemma 3.7. *If $b\#Z$ and $(u,A) \in \mathsf{plane}(Z)$ then $b\#A$ and $b \notin A$.*

Proof. By Lemma 3.6 $b\#A$ if and only if $b \notin A$. By definition $A = supp(u\|_{\emptyset} \cap Z)$. By Theorem 2.4 $supp(A) \subseteq supp(u\|_{\emptyset}) \cup supp(Z)$. It is easy to verify that $supp(u\|_{\emptyset}) = \emptyset$. The result follows. □

Theorem 3.8 ($(\mathbf{ren}{\mapsto})$). *If $b\#z$ then $z[a{\mapsto}b] = (b\ a)z$.*

Proof. We work by ϵ-induction. The case of $z \in \mathbb{A}$ is easy. So suppose $Z \notin \mathbb{A}$ (we adhere to our notational convention and write capital 'Z'). Suppose the inductive hypothesis of all $u \in Z$. By definition

$$Z[a{\mapsto}b] = \bigcup\{(u[a{\mapsto}b])\|_{A(a{\mapsto}\{b\})\backslash\delta(u,a,b)} \mid (u,A) \in \mathsf{plane}_{\{a,b\}}(Z)\}$$

Suppose $(u,A) \in \mathsf{plane}_{\{a,b\}}(Z)$, so $supp(u) \cap \{a,b\} \subseteq supp(u) \cap A$. By Lemma 3.7 we know $b \notin A$ and so $b\#u$. By inductive hypothesis $u[a{\mapsto}b] = (b\ a)u$, and by Theorem 2.3 $supp(u[a{\mapsto}b]){=}(b\ a)supp(u)$. Then we can simplify:

$$A(a{\mapsto}\{b\}) \setminus (supp(u)(a{\mapsto}\{b\}) \setminus supp(u[a{\mapsto}b]))$$
$$= ((b\ a)A) \setminus ((b\ a)supp(u) \setminus (b\ a)supp(u)) = (b\ a)A.$$

The result follows by Theorem 2.1 and the fact that the permutation action is pointwise. □

Lemma 3.9. $supp(u\|_A) \subseteq A$.

Proof. Consider any $\pi \in \mathsf{fix}(A)$. Then $\pi(u\|_A) = (\pi u)\|_A$ by Theorem 2.1. The result follows since $\mathsf{fix}(A)$ is a group.

Lemma 3.10. *If* $b\#x, Z$ *then*

$$Z[a{\mapsto}x] = \bigcup\{(u[a{\mapsto}x])\|_{A(a{\mapsto}supp(u))\setminus\delta(u,a,x)} \mid (u,A) \in \mathsf{plane}_{supp(x)\cup\{a,b\}}(Z)\}.$$

(Notice the extra b on the far right subscript.)

Proof. Suppose $(u,A) \in \mathsf{plane}(Z)$ and $supp(u) \cap (supp(x) \cup \{a\}) \subseteq supp(u) \cap A$. Note that $b \notin A$ by Lemma 3.7. Choose some fresh c (so $c\#u, A, x, Z$). By Theorem 2.1 $((c\ b)u, A) \in \mathsf{plane}(Z)$ and by Theorem 2.3 $supp((c\ b)u) \cap A = supp(u) \cap A$ and

$$supp((c\ b)u) \cap (supp(x) \cup \{a\}) = supp(u) \cap (supp(x) \cup \{a\}).$$

So if we show that

$$u[a{\mapsto}x]\|_{A(a{\mapsto}supp(x))\setminus\delta(u,a,x)} = ((c\ b)u)[a{\mapsto}x]\|_{A(a{\mapsto}supp(x))\setminus\delta((c\ b)u,a,x)}$$

then we are done. By Theorem 2.1 and the fact that $c, b\#x$, we know that $\delta((c\ b)u,a,x) = (c\ b)\delta(u,a,x)$. Also $c, b\#A$ by Lemma 3.7, so $A(a{\mapsto}supp(x)) \setminus \delta((c\ b)u,a,x) = A(a{\mapsto}supp(x)) \setminus \delta(u,a,x)$ by elementary set calculations. Also we know by Theorem 2.1 that $((c\ b)u)[a{\mapsto}x] = (c\ b)(u[a{\mapsto}x])$. The result now follows since the permutation action is pointwise, and by Lemma 3.9. □

Theorem 3.11 ((α)). *If* $b\#z$ *then* $z[a{\mapsto}x] = ((b\ a)z)[b{\mapsto}x]$.

Proof. It suffices to prove that if $b\#z$ then $z[a{\mapsto}x] \subseteq ((b\ a)z)[b{\mapsto}x]$; given that we obtain the equality by considering the set inclusion in the opposite direction, using Theorem 2.3 to note that $a\#(b\ a)z$. We work by ϵ-induction. The case where $z \in \mathbb{A}$ is easy, so suppose $Z \notin \mathbb{A}$ (we adhere to our convention and write capital 'Z') and suppose the inductive hypothesis of every $u \in Z$. Using Lemma 3.10 for the first equality and the definition of substitution for the second,

$$Z[a{\mapsto}x] = \bigcup\{(u[a{\mapsto}x])\|_{A(a{\mapsto}supp(x))\setminus\delta(u,a,x)} \mid (u,A) \in \mathsf{plane}_{supp(x)\cup\{a,b\}}(Z)\}$$

$$((b\ a)Z)[b{\mapsto}x] = \bigcup\{(u'[b{\mapsto}x])\|_{A'(b{\mapsto}supp(x))\setminus\delta(u',b,x)} \mid (u',A') \in \mathsf{plane}_{supp(x)\cup\{b\}}((b\ a)Z)\}.$$

Suppose $(u,A) \in \mathsf{plane}_{supp(x)}(Z)$. To prove our set inclusion we just need to exhibit some $(u',A') \in \mathsf{plane}_{supp(x)\cup\{b\}}((b\ a)Z)$ such that

$$u[a{\mapsto}x]\|_{A(a{\mapsto}supp(x))\setminus\delta(u,a,x)} = u'[b{\mapsto}x]\|_{A'(b{\mapsto}supp(x))\setminus\delta(u',b,x)}.$$

We choose $u' = (b\ a)u$ and $A' = (b\ a)A$. Then $(u',A') \in \mathsf{plane}((b\ a)Z)$ by Theorem 2.1. Also by definition of $\mathsf{plane}_{supp(x)\cup\{a,b\}}(Z)$ we know that

$$supp(u) \cap (supp(x) \cup \{a,b\}) \subseteq supp(u) \cap A.$$

Now $b \notin A$ by Lemma 3.7 and $b \in supp(x) \cup \{a, b\}$. Therefore $b\#u$. It is now not hard to use Theorem 2.3 and some elementary set calculations to calculate that

$$supp(u') \cap (supp(x) \cup \{b\}) \subseteq supp(u') \cap A'.$$

So $(u', A') \in \mathsf{plane}_{supp(x)\cup\{b\}}(Z)$.

Also since $b\#u$ by the inductive hypothesis $u'[b{\mapsto}x] = u[a{\mapsto}x]$. It remains to show $A(a{\mapsto}supp(x)) \setminus \delta(u, a, x) = ((b\ a)A)(b{\mapsto}supp(x)) \setminus \delta((b\ a)u, b, x)$. Now $A(a{\mapsto}supp(x)) = ((b\ a)A)(b{\mapsto}supp(x))$ if $b \notin A$ is a fact easily verified. Also

$$\delta((b\ a)u, b, x) = supp((b\ a)u)(b{\mapsto}supp(x)) \setminus supp(((b\ a)u)(b{\mapsto}supp(x)).$$

Now $supp((b\ a)u)(b{\mapsto}supp(x)) = supp(u)(a{\mapsto}supp(x))$ follows since $b\#u$, and $((b\ a)u)(b{\mapsto}supp(x)$ follows by the inductive hypothesis, and we are done. □

Theorem 3.12 **($(\#{\mapsto})$).** *If $a\#z$ then $z[a{\mapsto}x] = z$.*

Proof. We work by ϵ-induction. The case where $z \in \mathbb{A}$ is easy, so suppose $Z \notin \mathbb{A}$ (we adhere to our notational convention and write capital 'Z') and $a\#Z$ and suppose the inductive hypothesis of all $u \in Z$. By definition

$$Z[a{\mapsto}x] = \bigcup\{(u[a{\mapsto}x])\|_{A(a\mapsto supp(x))\setminus\delta(u,a,x)} \mid (u, A) \in \mathsf{plane}_{supp(x)\cup\{a\}}(Z)\}.$$

Consider any $(u, A) \in \mathsf{plane}_{supp(x)\cup\{a\}}(Z)$. By assumption $supp(u) \cap (supp(x) \cup \{a\}) \subseteq supp(u) \cap A$. By Lemma 3.7 $a\#A$. Therefore $a\#u$ and by inductive hypothesis $u[a{\mapsto}x] = u$. It remains only to prove that $A(a{\mapsto}supp(x)) \setminus \delta(u, a, x) = A$. This follows by elementary set calculations and the inductive hypothesis. □

Theorem 3.13 **($(\mathbf{abs}{\mapsto})$).** *If $c\#x$ then $([c]z)[a{\mapsto}x] = [c](z[a{\mapsto}x])$.*

Proof. If $a\#z$ then by Theorem 2.4 also $a\#[c]z$ and $([c]z)[a{\mapsto}x] = [c](z[a{\mapsto}x]) = [c]z$ follows by Theorem 3.12. So suppose $a \in supp(z)$. It is not hard to verify that

$$\mathsf{plane}([c]z) = \{((c', (c'\ c)z), supp(z) \setminus \{c\}) \mid c'\#z\} \cup \{((c, z), supp(z) \setminus \{c\})\},$$

and using this we can calculate the following facts:

$$\begin{aligned} z &= (c, z)\|_{supp(z)\setminus\{c\}} \\ ([c]z)[a{\mapsto}x] &= (c, z[a{\mapsto}x])\|_{(supp(z)\setminus\{c\})(a\mapsto supp(x))\setminus\delta((c,z),a,x)} \\ [c](z[a{\mapsto}x]) &= (c, z[a{\mapsto}x])\|_{supp(z[a\mapsto x])\setminus\{c\}} \end{aligned}$$

So it suffices to verify that

$$(supp(z) \setminus \{c\})(a{\mapsto}supp(x)) \setminus \delta((c, z), a, x) = supp(z[a{\mapsto}x]) \setminus \{c\}.$$

Now $\delta((c, z), a, x) = (supp(z) \cup \{c\})(a{\mapsto}supp(x)) \setminus (supp(z[a{\mapsto}x]) \cup \{c\})$ (we use Lemma 3.5 to calculate the support of a pairset) and the result follows by set calculations. □

Now for the main technical theorem of this paper:

Theorem 3.14. *Definition 3.4 is equivariant and satisfies* (α)*,* $(\#\mapsto)$*,* $(\mathbf{var}\mapsto)$*,* $(\mathbf{ren}\mapsto)$*, and* $(\mathbf{abs}\mapsto)$ *from Subsection 3.1.*

Proof. Equivariance is automatic by Theorem 2.1. $(\mathbf{var}\mapsto)$ is direct from the definition. Each of (α), $(\#\mapsto)$, $(\mathbf{ren}\mapsto)$, and $(\mathbf{abs}\mapsto)$ is by one of the theorems proved above. □

3.4 The rule $(\mathbf{F}\mapsto)$

Theorem 3.14 omits rule $(\mathbf{F}\mapsto)$. We must be careful; take

$$F(x,y)=\emptyset \quad (x=y) \qquad F(x,y)=\{\emptyset\} \quad (x\neq y).$$

Then $F(a,b)[b\mapsto a]=\{\emptyset\}[b\mapsto a]=\{\emptyset\}$ but $F(a[b\mapsto a],b[b\mapsto a])=\emptyset\neq\{\emptyset\}$.

Some functions in Fraenkel-Mostowski sets inspect names — and some do not. Call a function that satisfies $(\mathbf{F}\mapsto)$ **variable-like**, otherwise call it **name-like**. So what functions are variable-like?

Lemma 3.15. *If* $Z\notin\mathbb{A}$ *and* Z *is a* finite *set then* $Z[a\mapsto x]=\{z[a\mapsto x]\mid z\in Z\}$.

Proof. By definition,

$$Z[a\mapsto x]=\bigcup\{(u[a\mapsto x])\rVert_{A(a\mapsto supp(x))\setminus\delta(u,a,x)}\mid (u,A)\in \mathsf{plane}_{supp(x)\cup\{a\}}(Z)\}$$

Suppose $(u,A)\in\mathsf{plane}_{supp(x)\cup\{a\}}(Z)$. Since $u\rVert_A\subseteq Z$ and Z is finite, $u\rVert_A$ is finite. It follows that $supp(u)\subseteq A$ and $u\rVert_A=\{u\}$. It follows by set calculations that

$$supp(u[a\mapsto x])\subseteq A(a\mapsto supp(x))\setminus\delta(u,a,x)$$

and the result follows. □

Let Λ be the datatype of abstract syntax defined inductively by

$$\frac{a\in\mathbb{A}}{a\in\Lambda} \qquad \frac{x,y\in\Lambda}{(x,y)\in\Lambda} \qquad \frac{a\in\mathbb{A}\quad x\in\Lambda}{[a]x\in\Lambda}$$

It is not hard to show that Λ is isomorphic to λ-terms up to α-equivalence [13]; this is the standard construction of abstract-syntax-with-binding in Fraenkel-Mostowski sets, slightly modified (see the comment 'wrapped up' below). Atoms have zero elements, pairs have two, and abstractions have infinitely many, so no atom is a pair or an abstraction, and no pair is an abstraction.

Corollary 3.16. *The functions* $(x,y)=\{\{x\},\{x,y\}\}$*,* $\mathsf{inl}(x)=(x,\emptyset)$*, and* $\mathsf{inr}(x)=(x,\{\emptyset\})$*, all satisfy* $(\mathbf{F}\mapsto)$*. Also, all constants* $F()$ *satisfy* $(\mathbf{F}\mapsto)$*.*

Proof. The results for pairing, inl, and inr follow from Lemma 3.15. For the case of $F()$, by Theorem 2.4 we know $a\#F()$ so $F()[a\mapsto x]=F()$ by Theorem 3.12. □

Theorem 3.17. *If $z, x \in \Lambda$ then $z[a{\mapsto}x] \in \Lambda$ and $z[a{\mapsto}x]$ is equal to what we would normally call 'capture-avoiding substitution of x for a in z'.*

Proof. We work by induction on Λ.

- $a[a{\mapsto}x] = x$ and $b[a{\mapsto}x] = b$.
- $(z_1, z_2)[a{\mapsto}x] = (z_1[a{\mapsto}x], z_2[a{\mapsto}x])$ by Corollary 3.16.
- $([c]z)[a{\mapsto}x] = [c](z[a{\mapsto}x])$ providing $c\#x$ by Theorem 3.13 (it is not hard to prove inductively that $c\#x$ corresponds precisely to 'c is not free in x' when $x \in \Lambda$).

□

This theorem is generic and works for any datatype of syntax, though we must interpret atoms as themselves and not 'wrapped up' in the following sense: our construction is an isomorphic version of the datatype from [13] given by:

$$\frac{a \in \mathbb{A}}{\mathsf{inl}(\mathsf{inl}(a)) \in \Lambda} \qquad \frac{x, y \in \Lambda}{\mathsf{inl}(\mathsf{inr}((x, y))) \in \Lambda} \qquad \frac{a \in \mathbb{A} \quad x \in \Lambda}{\mathsf{inr}([a]x) \in \Lambda}$$

This is not suitable for Theorem 3.17 because atoms are wrapped up in $\mathsf{inl}(\mathsf{inl}(a))$ and $\mathsf{inl}(\mathsf{inl}(a))[a{\mapsto}x] = \mathsf{inl}(\mathsf{inl}(x)) \neq x$. Fraenkel-Mostowski sets substitution cannot anticipate our particular implementation of the tree-structure of datatypes; since atoms are a distinct class of elements in the Fraenkel-Mostowski set universe it is valid to insert them 'unwrapped' into the inductive type as we have done.

Corollary 3.18. *If $a\#y$ and $x, y, z \in \Lambda$ then $z[a{\mapsto}x][b{\mapsto}y] = z[b{\mapsto}y][a{\mapsto}x[b{\mapsto}y]]$.*

Proof. By induction on z. □

Corollary 3.18 is not necessarily true if $z \notin \Lambda$. For example if $a, b, c, d, e \in \mathbb{A}$ then

$$\begin{aligned}
&\{a, (a,a), (c,c), (d,d), \ldots\}[a{\mapsto}(b,b)][b{\mapsto}(c,c)] \\
&\qquad = (a\|_{\{a\}} \cup (d,d)\|_{\{b\}})[a{\mapsto}(b,b)][b{\mapsto}(c,c)] \\
&\qquad = (\{(b,b)\} \cup (d,d)\|_{\{b\}})[b{\mapsto}(c,c)] \\
&\qquad = (d,d)\|_{\emptyset}[b{\mapsto}(c,c)] \\
&\qquad = \{(a,a), (b,b), (c,c), \ldots\}
\end{aligned}$$

$$\begin{aligned}
&\{a, (a,a), (c,c), (d,d), \ldots\}[b{\mapsto}(c,c)][a{\mapsto}((c,c),(c,c))] \\
&\qquad = (a\|_{\{a\}} \cup (d,d)\|_{\{b\}})[b{\mapsto}(c,c)][a{\mapsto}((c,c),(c,c))] \\
&\qquad = (a\|_{\{a\}} \cup (d,d)\|_{\{c\}})[a{\mapsto}((c,c),(c,c))] \\
&\qquad = \{(a,a), (b,b), (d,d), (e,e), \ldots, ((c,c),(c,c))\}.
\end{aligned}$$

Corollary 3.18 states that substitution is variable-like on datatypes of syntax-with-binding (that means all the usual notions of substitution on syntax). Fraenkel-Mostowski set theory can also express name-like functions. Any substitution action makes a set z into a function over the atoms in its support, so that z can be viewed as representing

each of the functions 'x maps to $z[a{\mapsto}x]$' for each $a \in \mathit{supp}(z)$. The counterexample above shows that complex sets can become name-like. There is no conflict with the usual expectations for substitution; the sets concerned display structure beyond the reach of syntax.

Let us now return to the issue of canonicity. More than one substitution action on FM sets satisfies Definition 3.4. Can we identify some narrower class of substitution actions of which $[a{\mapsto}x]$ is a member and $[a{\mapsto}x]_t$ is not? The following property generalises $(\mathbf{abs}{\mapsto})$ and has a proof similar to that of Theorem 3.13:

Theorem 3.19. *If* $A \in \mathcal{P}_{\mathit{fin}}\mathbb{A}$ *and* $\mathit{supp}(x) \cap (\mathit{supp}(u) \setminus A) = \emptyset$ *then* $u\|_A[a{\mapsto}x] = u[a{\mapsto}x]\|_A$.

Any FM set can be decomposed as a (not necessarily disjoint) union of sets of the form $u\|_A$.[4] Our substitution action is such that if $Z = \bigcup_i z_i\|_{A_i}$ then $Z[a{\mapsto}x] = \bigcup_i z_i[a{\mapsto}x]\|_{A'_i}$ — subject to a host of sophisticated capture-avoidance issues which make up the technical meat of the definition.

This generalises $(\mathbf{abs}{\mapsto})$, but it is a profoundly infinitely property (in the decomposition, and also in the capture-avoidance conditions) and no syntax has yet been developed to express it as a neat equality between terms.

The remarkable fact is that we obtained $[a{\mapsto}t]$ *just* by satisfying this one property, and it then satisfies *all* the axioms of substitution from Definition 3.4, even though those axioms all look very different.

It is a fact that the author has developed another 'good' substitution action (the one outlined in this paper is the shorter and arguably the sweeter). As we mentioned above, a substitution action turns FM sets into a computational structure (of as-yet unknown power). Different substitution actions correspond to different notions of computation. A similar situation exists in the λ-calculus; β-reduction looks simple and specific but in fact it does not pin down a single notion of computation since many strategies exist to apply it (lazy, eager, and so on). There is also freedom to include other rules, such as η-reduction and η-expansion, or additional constants with their own special reduction rules. β-reduction *qua* rule is only the beginning of a larger story. The field of logic has a similar situation; there is classical truth, intuitionistic truth, relevant truth, many-valued truth, fuzzy truth, and so on. A certain amount of core behaviour is pinned down but useful latitude also remains. It is a fact that on sets representing actual syntax, the behaviour of substitution is fixed; but elsewhere latitude exists.

It would be interesting to pin down precisely the notion of computation given by the substitution action, and in particular to discover whether it is Turing complete.

4 Related work and Conclusions

4.1 Related work

CIAO Prolog [14, Extra-logical predicates] and a-logic [11] include a predicate $\mathsf{var}(x)$ which is true of x when x is a variable symbol (its use is to guide proof-search). The

[4] Not every decomposition of sets is suitable. For example $\mathbb{A} = \{a\} \cup (\mathbb{A} \setminus \{a\})$ but we do not want to base a substitution action on this decomposition since $a\#\mathbb{A}$ but $\neg a\#\{a\}$ and $\neg a\#(\mathbb{A} \setminus \{a\})$.

functional programming language CURRY [15] has similar constructs. These 'non-standard' functions behave like F that are *not* variable-like, and so current denotational methods find them hard to account for.

Fine [9] has undertaken a philosophical axiomatic investigation of *dependency between objects*. A connection may be established if we take substitution to give a notion of 'correlation' between sets; for example if $z[a{\mapsto}x] = z'$, or $z[a{\mapsto}x] = z'[a{\mapsto}x]$, then z and z' are 'correlated'.

Aczel's 'generalised set theory' [2] and 'universes with parameters' [3] model variable symbols (Aczel calls them *parameters*) as atoms in a ZFA-like set theory. The resemblance ends there; we discover structure inherent in a known and relatively simple set theory, whereas Aczel imposes all the structure he needs as extra axioms on names. Aczel's programme is also different and concerns itself with coinductive structures and non-wellfounded set theory [1] as a semantics for behaviour. Aczel's substitution action is not capture-avoiding, in the sense that $Z[a{\mapsto}x] = \{z'[a{\mapsto}x] \mid z' \in Z\}$ always, like the naïve substitution action $[a{\mapsto}x]_n$ from Subsection 3.2. .

Nominal terms [23] come equipped with a substitution action, but this is for *context* variables and is not capture-avoiding. A capture-avoiding substitution action has been axiomatised using equalities between nominal terms [12]; our Theorem 3.14 can be read as stating that the construction in this paper validates those axioms (but it is not built out of syntax).

Other axioms for substitution exist; for example in work by Salibra [19], by Beeson [4], by Crabbé [7], and by Feldmann [8]. The axioms of Crabbé and Feldmann are not of capture-avoiding substitution. The axioms of Salibra and Beeson are. Salibra is interested in models but his models are abstract (he does universal algebra). Beeson is interested in theorem-proving. The axioms of Salibra and Beeson for substitution are embedded in larger theories of Turing-complete computation based on the λ-calculus, which gives their treatment a very different flavour from ours. Feldman's axioms are based on a concrete model using *environments* (functions mapping variables to denotational objects) which is very different from our model which is based on a set-theory (a far larger construction) and in which variables inhabit the denotation in a more direct sense.

Schmidt-Schauß and Siekmann introduce unification algebra [20] and Williams introduces instantiation theory [24]. In both cases an axiomatic approach is taken to substitution. The idea is to free the theory of unification from (the shackles of) syntax and give an axiomatic account of instantiation and unification. The axioms phrase abstract versions of instantiation and unification as known in syntax; in this paper we have presented a concrete model of an entirely *independent structure* (the axioms of set theory) which is *also* a model of substitution.

The work by Schmidt-Schauß, Siekmann, and Williams, all involve 'bespoke' (axiomatic) semantics which build in the basic elements of the object of study. Aczel's set theories [2, 3] also contain elements clearly identifiable as variables along with dedicated axioms clearly tailor-made to endow them with substitution. Abstraction and capture-avoidance, where these are considered at all, are always 'bespoke'.

4.2 Conclusions

In this paper we have discovered a capture-avoiding substitution action, defined purely by ϵ-induction on set structure, and hiding in a set theory known for the better part of a century [5, 22]. This raises philosophical questions about substitution, so familiar as a concrete operation on syntax.

The FM model displays substitution behaviour spontaneously and not because of any obvious design to 'make it work'; the axioms of FM sets have no evident features to suggest the application we have made of it in this paper. Indeed the theory was designed with a quite different application in mind, and even after the author and Pitts re-discovered it and applied it to syntax-with-binding [13], the substitution action remained hidden for several years.

The existence of universes of sets with a substitution action raises some interesting prospects.

FM sets contains sets representing syntax and sets representing semantics and the set theory marks no barrier between them. Substitution extends smoothly from sets representing syntax to all sets. Can we extend syntax-based proof-methods along this substitution action to denotation; for example by using ϵ-induction principles on sets of denotations with their substitution action, like we use inductive arguments on syntax? Can we use FM sets as an arena to consider the 'non-standard' calculi and logics mentioned in the Related Work? Is it possible to now consider FM semantics for well-understood systems such as name-passing calculi [17]?

We should try to understand the computational content of the substitution action; a set z can be considered a function over the atoms in its support, with 'z applied to x' interpreted as $z[a{\mapsto}x]$ (we get a different function for each $a \in \mathit{supp}(z)$). What is the precise power of this notion of computation; is it Turing complete?

If we have a sets universe with a substitution action we can consider all the things that are usually done with syntax (and arguably *trapped* in syntax) but now we can give them a more abstract meaning; for example we can consider unification, matching, rewriting, and to some extent solutions to simultaneous equations — on Fraenkel-Mostowski sets. This raises the intriguing possibility that giving a set-theoretic semantics for example to rewriting systems may have a similar significance as Scott's set-theoretic semantics of the λ-calculus [21].

We hope this work will stimulate a different view of variables in syntax and semantics and open new possibilities in both.

References

[1] Peter Aczel. *Non-wellfounded Set Theory*. Number 14 in CSLI lecture notes. CSLI, 1988.

[2] Peter Aczel. Generalised set theory. *CSLI lecture notes*, 1(58):1–17, 1996.

[3] Peter Aczel and Rachel Lunnon. Universes and parameters. *CSLI lecture notes*, 2:3–24, 1991.

[4] Michael Beeson. Lambda logic. In *Second International Joint Conference on Automated Reasoning (IJCAR 2004)*, volume 3097 of *LNCS*, pages 460–474. Springer, 2004.

[5] N. Brunner. 75 years of independence proofs by fraenkel-mostowski permutation models, 1996.

[6] James Cheney and Christian Urban. Alpha-prolog: A logic programming language with names, binding and alpha-equivalence. In Bart Demoen and Vladimir Lifschitz, editors, *Proceedings of the 20th International Conference on Logic Programming (ICLP 2004)*, number 3132 in LNCS, pages 269–283. Springer-Verlag, September 2004.

[7] M. Crabbé. On the notion of substitution. *Logic Journal of the IGPL*, 12 n.2:111–124, 2004.

[8] Norman Feldman. Axiomatization of polynomial substitution algebras. *Journal of Symbolic Logic*, 47(3):481–492, September 1982.

[9] Kit Fine. *Reasoning with Arbitrary Objects*. Blackwell, 1985.

[10] Murdoch J. Gabbay. *A Theory of Inductive Definitions with alpha-Equivalence*. PhD thesis, Cambridge, UK, 2000.

[11] Murdoch J. Gabbay and Michael J. Gabbay. a-logic. In *We Will Show Them: Essays in Honour of Dov Gabbay*, volume 1. College Publications, 2005.

[12] Murdoch J. Gabbay and Aad Mathijssen. Capture-avoiding substitution as a nominal algebra. In *ICTAC'2006*, 2006.

[13] Murdoch J. Gabbay and A. M. Pitts. A new approach to abstract syntax with variable binding. *Formal Aspects of Computing*, 13(3–5):341–363, 2001.

[14] CLIP group. CIAO Prolog documentation project. `http://www.ciaohome.org`.

[15] Michael Hanus, Sergio Antoy, Herbert Kuchen, Francisco J. López-Fraguas, Wolfgang Lux, Juan José Moreno-Navarro, and Frank Steiner. Curry: An integrated functional logic language. April 2003.

[16] P. T. Johnstone. *Notes on logic and set theory*. Cambridge University Press, 1987.

[17] Robin Milner, Joachim Parrow, and David Walker. A calculus of mobile processes, II. *Information and Computation*, 100(1):41–77, September 1992.

[18] A. M. Pitts and Murdoch J. Gabbay. A metalanguage for programming with bound names modulo renaming. In R. Backhouse and J. N. Oliveira, editors, *Mathematics of Program Construction. 5th International Conference, MPC2000, Ponte de Lima, Portugal, July 2000. Proceedings*, volume 1837 of *Lecture Notes in Computer Science*, pages 230–255. Springer-Verlag, Heidelberg, 2000.

[19] Antonino Salibra. On the algebraic models of lambda calculus. *Theoretical Computer Science*, 249(1):197–240, 2000.

[20] Manfred Schmidt-Schauß and Jörg Siekmann. Unification algebras: An axiomatic approach to unification, equation solving and constraint solving. Technical Report SR-88-09, FB Informatik, Kaiserslautern, 1988.

[21] Joseph E. Stoy. *Denotational Semantics: The Scott-Strachey Approach to Programming Language Theory*. MIT Press, Cambridge, MA, USA, 1977.

[22] J. Truss. Permutations and the axiom of choice. In H. D. Macpherson R. Kaye, editor, *Automorphisms of first order structures*, pages 131–152. Oxford University Press, 1994.

[23] C. Urban, A. M. Pitts, and Murdoch J. Gabbay. Nominal unification. *Theoretical Computer Science*, 323(1–3):473–497, 2004.

[24] James Williams. Instantiation theory. Technical Report M-90-04, The Mitre Corporation, 1990.